

Embedded Systems, Cyber-Physical Systems and Robotics Project Report

NeuroDrive: EEG-Based Smart Vehicle

Submitted by

Herry Patel
Bhumik Fadadu
Rajkumar Kakadiya
Janmdeepsinh Vala
Kashyap Limbasiya
Darshit Dobariya
Ayush Sakariya
Jeet Upadhyay

Submitted on

September 9, 2026

Abstract

NeuroDrive is a ground vehicle driven by a consumer EEG headset. A NeuroSky MindWave Mobile 2 streams attention and signal-quality metrics over Bluetooth to a laptop, and a webcam watches the driver. A Python bridge conditions both inputs and transmits one of four discrete commands (plus a keepalive ping) over WiFi to an ESP32, which runs a four-state machine and drives a two-wheel chassis through an L298N H-bridge. Sustained concentration moves the vehicle forward, a raised hand steers it, and removing the headset stops it.

Making a vehicle move on concentration is not the engineering problem. That part is a threshold and a motor driver. The problem is that the control signal is slow, noisy and frequently absent, and that it is in charge of something with wheels. One premise therefore organises the architecture: stopping must be more reliable than starting. Four safety layers stand in series between a command and a turning wheel, a mechanical kill switch, a latching emergency-stop button, a two-second watchdog on the vehicle, and a signal-loss stop on the laptop. Each one fails for reasons the others do not share, and none depends on another being alive.

This report presents the design rather than a build log. It covers the pipeline, the electronics, the software structure, the state machine, the protocol, the driving policy and the safety argument, and for every decision that could reasonably have gone the other way it names the alternative and says why we rejected it. The protocol, the state machine and the module interfaces are fixed in a written contract, and the bridge runs against synthetic and recorded signal sources, so the whole control chain can be exercised on a bare laptop with no headset and no vehicle present.

Contents

1	Introduction	1
1.1	Motivation and problem	1
1.2	Objectives and contributions	1
2	Background and approach	3
2.1	Choosing what to read from the brain	3
2.2	What a single forehead electrode delivers	3
2.3	Sparse intent, and the gap this project fills	4
3	System design and architecture	5
3.1	Design goals and overview	5
3.2	Hardware architecture	6
3.3	Software architecture	6
3.4	The vehicle state machine	7
3.5	Communication	8
3.6	Control policy	9
3.7	Safety architecture	10
4	Use cases and applications	12
4.1	Where the approach transfers	12
4.2	Where it does not belong	12
5	Conclusion and future work	14
5.1	Summary and lessons	14
5.2	Limitations	14
5.3	Future work	15
5.4	Concluding remarks	15
	Bibliography	16

Chapter 1

Introduction

1.1 Motivation and problem

A brain–computer interface offers a control channel that bypasses the body. No hands, no voice, no line of sight. Consumer EEG hardware has brought that channel within reach of an ordinary student project, since a headset now costs less than a mid-range keyboard and is worn in seconds without preparation.

What is easy to underestimate is the distance between a signal on a screen and a signal driving a vehicle. A misclassification in a laboratory spelling task produces a wrong character. The same misclassification in a moving vehicle produces a vehicle moving in a direction nobody asked for, in a room with people in it. The problem this project addresses is therefore:

Given a single-channel consumer EEG headset reporting a coarse attention metric once per second, together with signal-quality information, build a ground vehicle a person can drive with that signal, and make the system’s behaviour under every foreseeable failure of headset, laptop, radio link or firmware be that the vehicle stops.

No single component is difficult. Reading a serial protocol, thresholding a number and driving an H-bridge are all unremarkable on their own. The difficulty lies in three properties of the chain as a whole.

The control signal was never designed for control. It is one scalar, arriving once per second, moving ten points between readings while the user’s state has not changed, and carrying no directional information at all.

Every link can fail silently. A lifted electrode, a dropped pairing, a sleeping laptop and a crashed process all look identical from downstream, because data simply stops arriving. A system that keeps executing its last instruction when that happens drives away from a user who has taken the headset off.

Correctness cannot be shown by driving the vehicle around. The behaviours that most need to be right are the watchdog, the emergency stop and the response to signal loss, and those are exactly the ones a successful demonstration never exercises.

1.2 Objectives and contributions

We set ourselves six objectives. Decode the headset stream reliably, including its failure indications. Condition it into a stable driving decision. Define a laptop-to-vehicle protocol that is easy to observe and safe when datagrams go missing. Implement the firmware as an

explicit state machine with an independent emergency stop and watchdog. Build safety from independent layers rather than one carefully written stop path. And make the whole chain testable without hardware.

What came out of it is a complete working system, described in Chapter 3. Its two parts worth borrowing are a safety architecture organised around independent failure modes rather than redundancy, and a frozen interface contract between the two halves, with a bridge that runs on synthetic and recorded signal sources when no hardware is present.

This report is about the design. Chapter 2 explains why the project reads concentration rather than imagined movement, and what a single forehead electrode can honestly deliver. Chapter 3 is the substance: the pipeline, the electronics, the software structure, the state machine, the protocol, the policy and the safety argument, with the rejected alternative named alongside each decision. Chapter 4 says where this approach transfers and where it does not, and Chapter 5 sets out what we would change.

Eight of us built it for the Embedded Systems, Cyber-Physical Systems and Robotics project seminar, split across hardware, firmware, signal processing, application software, testing and integration.

Chapter 2

Background and approach

2.1 Choosing what to read from the brain

Non-invasive brain–computer interfaces are grouped by the phenomenon they exploit, and that choice determines almost everything else about the system built on top. Motor imagery, where the user imagines moving a limb, is self-paced and needs no stimulus. It also needs several electrodes at specific scalp positions, a training session for each user, and a classifier. Steady-state visually evoked potentials give a high information rate with very little training, but they require the user to keep looking at a flickering display, which is a poor fit for someone who also has to watch where a vehicle is going. Event-related potentials such as P300 are accurate with minimal training, and the stimulus sequence paces them, which puts a floor under command latency. Passive metrics take a different route. They track a continuous cognitive state instead of classifying a discrete intent, so the information rate is low and the metric coarse, but they need one electrode, no stimulus and no training.[4, 5] We chose the passive-metric route for fit, not ambition. Four commands need very little information rate. What they do need is a control the user can operate while walking behind the vehicle and watching it, with no stimulus display and no classifier training before a demonstration.

2.2 What a single forehead electrode delivers

The MindWave carries one dry electrode at Fp1, referenced against an earlobe clip.[3] That position sits over the frontalis muscle and centimetres from the eyes, so jaw clenching, frowning and eye movement all produce potentials larger than the cortical signal underneath. A system built on this sensor is not reading pure cortical activity, and we would not try to defend the claim that it is. Two consequences shaped the design. First, the attention metric is fit for a coarse two-state decision and nothing finer, which is exactly how the driving policy uses it: concentrate to go, relax to stop. Second, and decisively, a scalar cannot express direction. One electrode reporting one number per second carries no left and no right, and no amount of conditioning will extract them from it. Steering therefore has to come from somewhere else. We take it from a webcam: the driver raises a hand, and the vehicle turns that way for as long as the hand stays up. The gesture is deliberate, unambiguous, and visible to everyone watching, which the EEG signal is not. The headset keeps the job it is actually good at, which is deciding whether the vehicle should be moving at all.

2.3 Sparse intent, and the gap this project fills

Work on brain-actuated wheelchairs and mobile robots has converged on shared control.[\[1, 2\]](#) The neural signal supplies sparse, high-level intent, and autonomous navigation supplies the continuous control that keeps the vehicle from hitting things. NeuroDrive applies that principle at the scale it actually operates. There is no autonomous navigation to share control with, and the vehicle carries no sensing of its own surroundings at all; the camera watches the driver, not the road. What it does instead is make the vehicle's behaviour in the absence of intent unambiguous and safe. Where a shared-control wheelchair fills the gaps between commands with autonomy, NeuroDrive fills them with a stop. That is where our contribution sits. Interface research reports accuracy, information transfer rate and task completion time. It says comparatively little about what a system does when the electrode lifts off, and less about verifying that behaviour repeatedly. For a system that moves under its own power, that is the property that matters most, and it is what this report is organised around.

Chapter 3

System design and architecture

3.1 Design goals and overview

Three properties of the input channel shape everything else. It is slow. Metrics arrive once per second, and no software makes intent visible sooner. It is noisy, and sometimes it is not there at all, because a dry electrode picks up muscle and eye activity along with cortical signal and reports a poor-signal byte the moment contact is lost. Finally, it cannot be aimed. A scalar from 0 to 100 has no way to express “turn left”, which is why steering uses a separate deliberate gesture and attention is left to decide only whether to go or stop.

The goals that follow are the ones this chapter keeps returning to. Stopping must be more reliable than starting. No component may be trusted to be alive. Retuning must not require editing code. And the system must be verifiable without hardware.

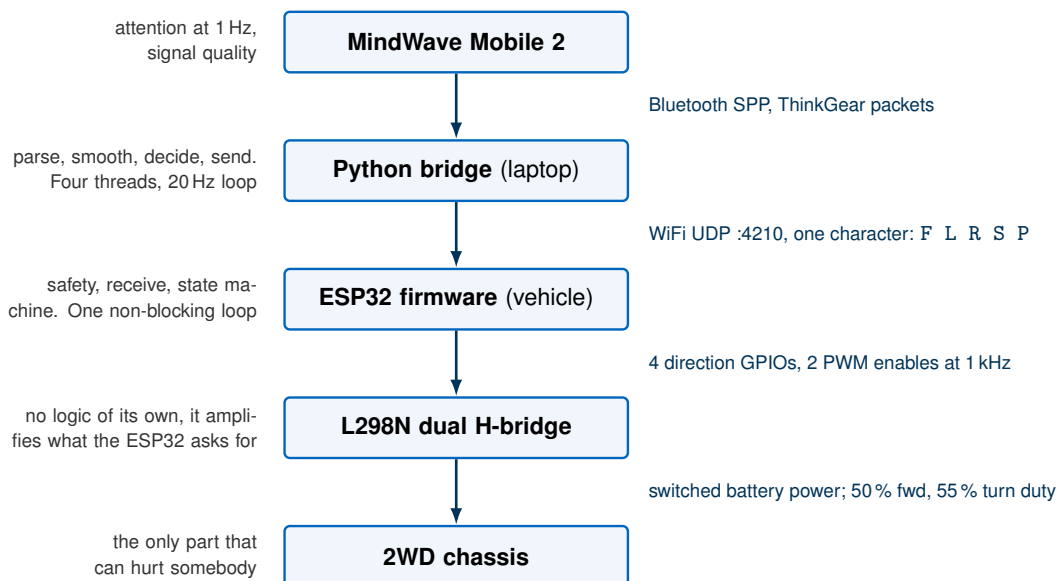


Figure 3.1: The NeuroDrive pipeline and its four interfaces. Every interface can be observed on its own: ThinkGear packets in a serial capture, commands in a network capture, PWM on an oscilloscope.

Two aspects of Figure 3.1 are choices rather than consequences.

The intelligence lives on the laptop. The ESP32 has no idea what EEG is. It receives F, L, R, S or P and nothing else. This keeps the safety-critical half small enough to hold in your head, about six hundred lines across four modules, accepting five characters and one button. It also means the driving policy can be retuned without reflashing anything.

The interfaces are narrow and frozen. We wrote the protocol, the state machine, the

module signatures and the pin map into one document that nobody changes alone. That document is what let eight people work in parallel without the two halves drifting apart.

3.2 Hardware architecture

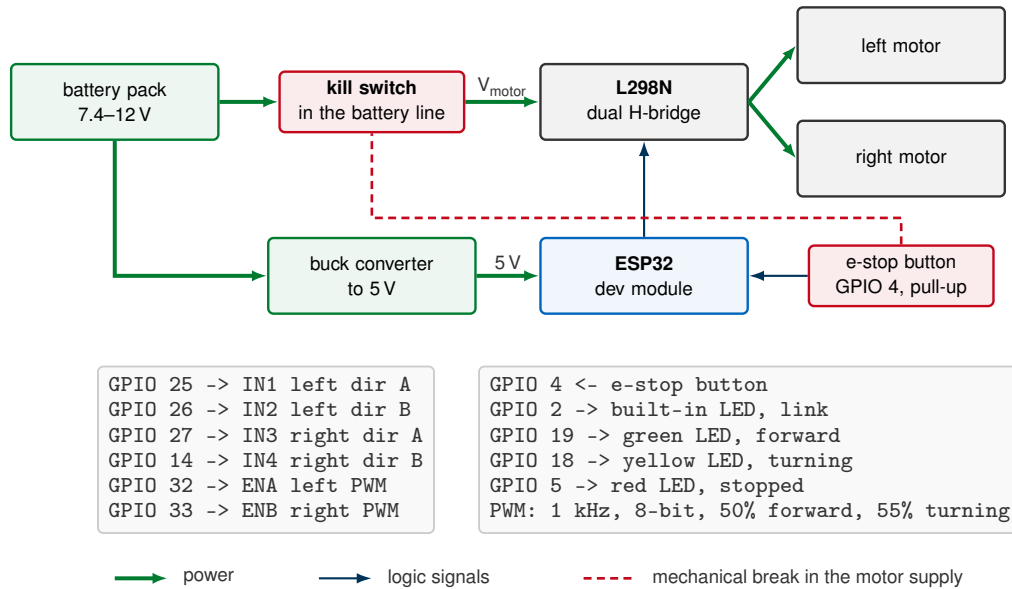


Figure 3.2: Electrical architecture and pin assignment. The emergency stop appears twice on purpose, as a signal the firmware reads and as a mechanical break in the battery line that no software failure can defeat.

The important feature of Figure 3.2 is that two power domains meet only at a common ground.

Motor current never passes through a logic pin. The battery feeds the L298N directly, and the ESP32 contributes only direction bits and PWM enables. An ESP32 pin sources tens of milliamps, a stalled geared motor draws over an amp, and the failure is not graceful. We stated the rule as an absolute, never, not once, not for a quick test, because that is what keeps it from being violated during a late-night debugging session.

Speed is duty cycle, not voltage. Both enable pins carry 8-bit PWM at 1 kHz, at 50 % forward and 55 % turning, because a pivot turn has to break static friction on both wheels at once. A per-side trim compensates for two nominally identical motors never being identical. A minimum duty floor stops the firmware ever asking for a duty too small to turn a wheel, which produces a buzz and a stationary vehicle rather than slow motion.

We picked the ESP32 for one reason that matters: WiFi on the die, so it can run as its own access point, broadcast its own network and always answer to 192.168.4.1. A demonstration that depends on a venue’s wireless network is one the venue can take away.

3.3 Software architecture

The laptop application is six modules with one responsibility each (Table 3.1), spread across four threads. We divided them along one axis: how likely is this code to change, and for what reason?

Table 3.1: Bridge modules, and the reason each one stands alone.

Module	Responsibility, and why it is separate
thinkgear	Packet framing and checksums. A pure function of a byte stream, so corrupt data needs no headset to exercise
eeg_sources	The acquisition interface, with serial, mock and replay implementations. A different headset is one new class
signal_processor	Conditioning only: smoothing and quality assessment. No driving policy
vision	Gesture detection only: pose landmarks, hold and refractory debouncing. Knows nothing about the vehicle
command_mapper	Driving policy only: thresholds, hysteresis, turn direction. No signal processing
wifi_sender	Transport, re-send scheduling, acknowledgements. Knows nothing about EEG

The split between the last two earns its keep almost every week. Detection parameters belong to the headset and the user; policy belongs to what we want the vehicle to do. Keeping them apart means an evening spent tuning thresholds never touches detection code.

Each thread exists because of a specific failure it prevents. A blocking read on a Bluetooth port can take hundreds of milliseconds, and on the control loop's thread that stall would delay the decision to stop. Camera capture and pose inference are slower still and run at their own frame rate, so they occupy a thread of their own for the same reason. The main thread runs the 20 Hz loop and owns all policy state, single-threaded, so the same samples and clock values always produce the same commands. The sender thread exists because a router hiccup blocking a socket write must never become time in which no EEG is processed.

The two hand-offs are deliberately asymmetric. Samples pass through a queue, because losing one loses information. Commands pass through a single-slot mailbox that overwrites, because a 20 Hz loop must not build a backlog of stale instructions that the vehicle then works through after the user has stopped concentrating.

3.4 The vehicle state machine

The firmware is four modules, `safety`, `comm`, `motor_control` and `status_led`, run in that order every iteration. Safety supervision runs first and unconditionally, so a pressed button or an expired watchdog has already stopped the vehicle before a newly arrived command can be acted on. `motor_control` owns every write to the L298N pins, and that is what makes the rest trustworthy. The emergency stop does not need to know which pins are high. It calls one function.

Three rules in Figure 3.3 encode decisions rather than mechanics.

A turn is a timed pulse, not a state you sit in. The vehicle turns only while the bridge keeps saying so, and the pulse expires on its own if it stops. Lowering the hand is therefore not a command the firmware has to receive and act on; it is simply the absence of one. A gesture misread as a turn costs 300 ms of deviation instead of a vehicle spinning until somebody intervenes.

A turn returns to the base state. F arriving mid-turn updates what resumes without interrupting the turn, while S stops immediately and also sets the base state, so a turn cannot resume forward motion after the user has asked to stop.

Re-entering a turn does not restart its timer. The bridge re-sends the current command as a keepalive, and without this rule those re-sends would push the expiry further out on every

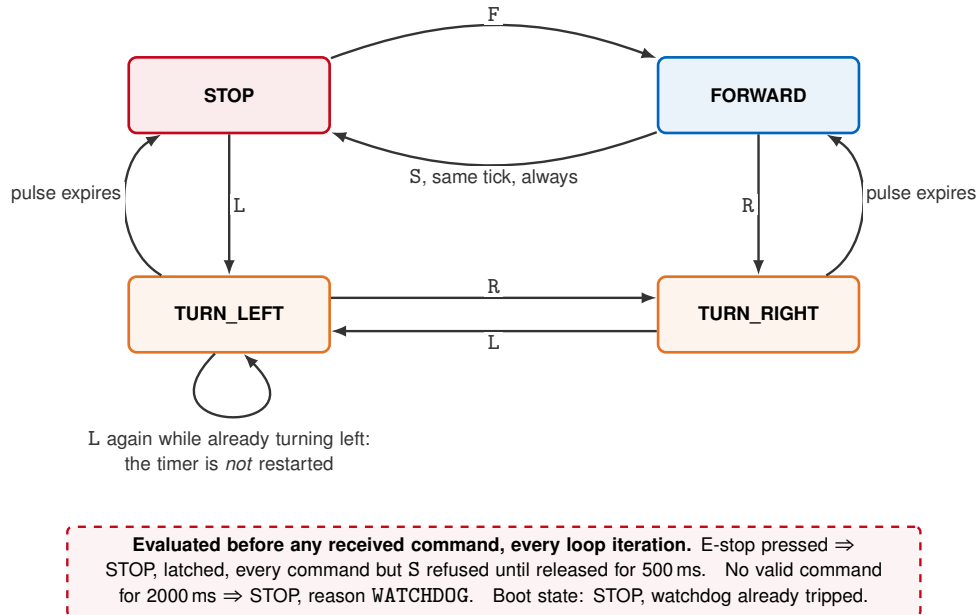


Figure 3.3: The vehicle state machine as implemented in the ESP32 firmware. A turn returns to the *base state*, whichever of STOP or FORWARD was commanded last. These four states and their transitions are what the interface contract fixes, so the bridge can reason about the vehicle's behaviour without observing it.

arrival. The firmware therefore refuses to restart a pulse that is already running, and a turn held longer than 300 ms arrives as a train of pulses rather than one indefinite state.

That is what makes a held gesture work. While the hand is up, the bridge keeps transmitting the turn and the firmware keeps re-entering the pulse, so the vehicle turns continuously; when the hand comes down the transmissions stop and the final pulse expires by itself. The two constants live in different files, one in a C header and one in a JSON file, and nothing but this paragraph keeps them consistent.

3.5 Communication

The protocol is one ASCII character terminated by a newline. F drives forward, L and R fire the two turn pulses, S stops, and P is a ping that feeds the watchdog and changes nothing else. The firmware also accepts the case-insensitive long forms. A binary encoding would buy nothing at five commands and would cost a great deal of debuggability. This format is readable in a serial monitor, visible in a packet capture, and typeable by hand when something is wrong at an inconvenient hour.

The transport is UDP on port 4210, with USB serial as a one-line fallback. TCP is the wrong tool here, for a specific reason rather than a general preference. A lost segment stalls everything behind it until it is retransmitted, and that is exactly wrong for commands whose value decays in milliseconds. A stop delivered reliably but late is worse than useless. UDP lets the application decide what to do about loss, and what the bridge does is send the current command again 250 ms later.

That periodic re-send doubles as the vehicle's watchdog keepalive, and it is the best decision in the communication design. Because ordinary traffic feeds the watchdog rather than a dedicated heartbeat, the link cannot look healthy while the bridge is dead. A lost datagram heals within 250 ms. A bridge that stops for any reason halts the vehicle within two seconds without having to announce anything.

The firmware acknowledges each accepted command after updating the state machine, so a round-trip measurement includes the vehicle reacting rather than just the network hop. Acknowledgements are advisory. The bridge never waits for one and never retransmits because one is missing, since making delivery conditional on them would reintroduce exactly the head-of-line blocking that choosing UDP avoided.

3.6 Control policy

The whole policy states in five lines. Smoothed attention at or above 60 means FORWARD. Below 40, sustained for more than one second, means STOP. Anything between holds the current state. A raised hand produces a turn in that direction, held for as long as the hand stays up. And no connection, no attention value yet, or poor signal quality means STOP unconditionally, before any of the above is considered.

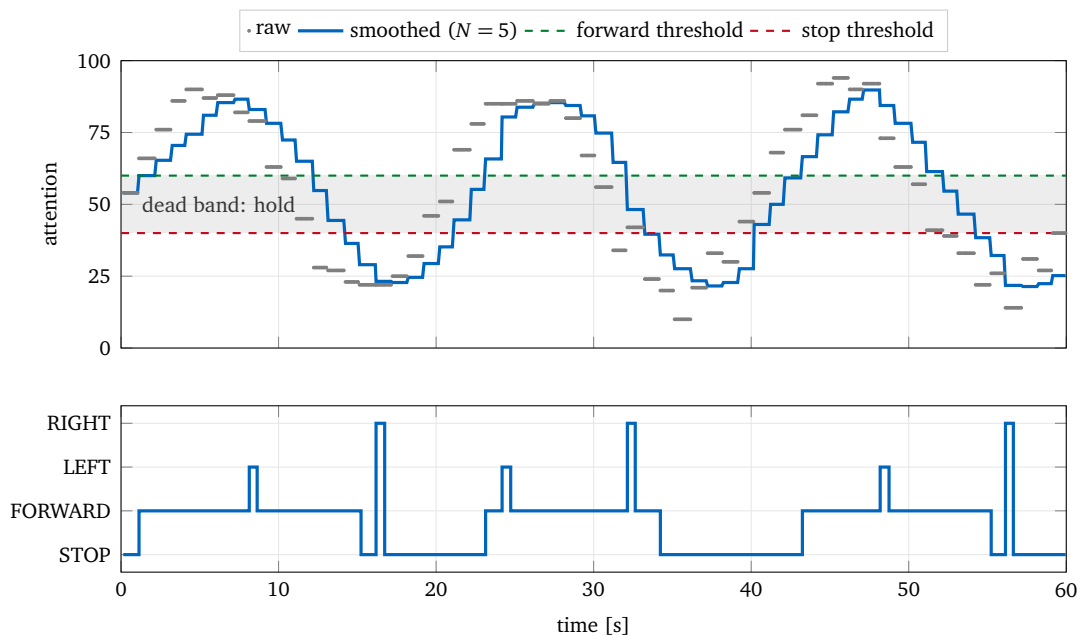


Figure 3.4: Sixty seconds of a recorded session, from the CSV the bridge writes on every run. The grey band is the dead band between the two thresholds, inside which the vehicle holds whatever state it is in.

Two of those rules are hysteresis wearing different clothes, and Figure 3.4 shows why both are needed. The dead band exists because a single threshold on a signal that swings ten points between readings gives you a vehicle starting and stopping several times a second while the user’s state has not changed at all. The one-second stop delay stops a single low sample from interrupting a deliberate drive.

The delay applies only to the attention-based stop. A lost link or poor signal quality stops the vehicle on the next cycle, with no hold whatsoever. We made that asymmetry on purpose. Hesitating is acceptable when the evidence is a noisy number and unacceptable when the evidence is that we no longer know what the user wants.

Turns come from the webcam: a raised right hand turns right, a raised left hand turns left. This is not a second, independent control path. Gestures fold into the same mapper and pass the same gates as everything else, so a raised hand cannot move a vehicle whose operator the headset has lost track of. If the headset comes off mid-run the vehicle stops, and waving at it will not restart it. This is a brain-controlled vehicle with a camera attached, not

a camera-controlled vehicle.

3.7 Safety architecture

Safety here is a property of how the components are arranged rather than a feature bolted on. Figure 3.5 draws the four layers the way they actually work, in series, so every one of them has to permit motion before a wheel turns.

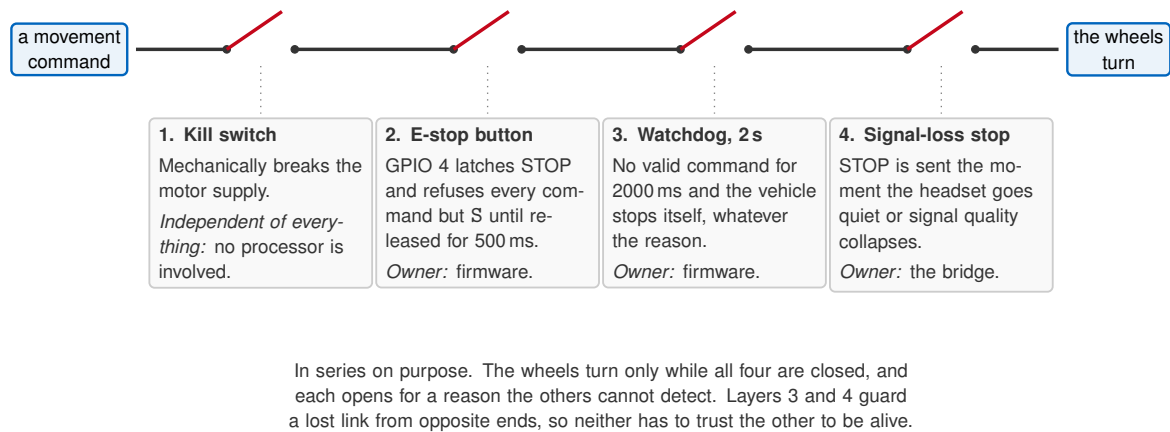


Figure 3.5: Four safety layers in series. What matters is not the number but that each one fails for reasons the others do not share.

The organising idea is independence, not redundancy. Three copies of the same software check fail together on the same bug. These four fail differently. Layer 1 has no processor in it. Layer 2 is firmware reacting to a physical input. Layer 3 is firmware reacting to the absence of input. Layer 4 is application software on another machine reacting to a different sensor entirely.

Layers 3 and 4 look redundant and are not. If the bridge crashes, its layer is gone but the vehicle's is not. If the vehicle's WiFi fails, the bridge's messages are gone but the watchdog is not. Neither needs the other to be working, which is precisely the property that redundant copies of one mechanism lack.

Two smaller decisions finish the picture. The vehicle boots into STOP with its watchdog already tripped, so there is no window after power-on in which it might move before the bridge has said anything. And the bridge transmits STOP on every exit path, including a clean quit, a keyboard interrupt and an unhandled exception, so closing the terminal ends a run rather than abandoning a moving vehicle.

Table 3.2 collects the decisions argued for above, each with the alternative we rejected. The pattern is consistent. We preferred the arrangement whose failure is visible and safe over the one that is more capable when everything works.

Table 3.2: Architectural decisions and the alternatives we rejected.

Decision	Alternative	Why
Policy on the laptop, state machine on the vehicle	All processing on the ESP32	Keeps safety-critical code readable in one sitting; re-tuning needs no reflash
One ASCII character over UDP	Binary framing over TCP	Debuggable by hand; loss self-heals in 250 ms instead of stalling
Re-send doubling as keepalive	A separate heartbeat	The link cannot look alive while the sender is dead
Turn as a timed pulse	A turn held until cancelled	A misread gesture costs 300 ms, not an unbounded spin
Attention drives, raised hands steer	Attention alone	A scalar cannot express direction; a hand raise is deliberate, unambiguous and visible to onlookers
Dead band and a one-second stop hold	A single threshold	The raw signal crosses any single threshold repeatedly per second
Four independent stop layers	One well-tested stop path	Independent failure modes; the outermost contains no software

Chapter 4

Use cases and applications

4.1 Where the approach transfers

Assistive mobility. What transfers from NeuroDrive is the safety architecture, not the control channel. A powered wheelchair that continues on its last command after the interface stops reporting is unacceptable in a way that scales with the mass of the vehicle.[1, 2] A watchdog fed by ordinary command traffic, an independent stop from the interface side, and a mechanical interruption that needs no software are the same three things at any scale. The hysteresis treatment of a noisy control signal transfers too, and particularly the asymmetry. Hesitate before stopping on a noisy number. Stop instantly on lost evidence.

Neurofeedback and attention training. Turning concentration into visible motion makes an internal state legible with very little explanation. Nobody needs the bar chart explained. Every session is recorded as a CSV of raw and smoothed attention, signal quality and the resulting command, so a session can be reviewed afterwards rather than only experienced. The thresholds are configuration, so the difficulty can be matched to a user.

Education and public engagement. The project's most defensible application is the one we built it for. It covers biosignal acquisition, packet parsing, signal conditioning, control policy, protocol design, embedded firmware, motor electronics and safety engineering, in a system a student can hold in two hands. Every layer can be observed on its own, and the whole thing runs without hardware, so a class of thirty can work on it with one headset between them. For an audience the projected dashboard does the real work. A vehicle that moves is a magic trick. A vehicle that moves beside a live attention bar, with the threshold marked and the reason for the current command spelled out in words, demonstrates a mechanism.

4.2 Where it does not belong

Anything safety-critical that relies on the intent channel. The four layers make the vehicle reliably stop. Nothing in the system makes it reliably go where the user intends. A single-channel forehead electrode reporting a proprietary scalar once per second is not a trustworthy source of intent, and we built the architecture so that this untrustworthy channel can only cause motion the trustworthy layers can arrest.

Any inference about the user beyond the driving decision. The system logs attention and meditation values. Presenting these as measurements of a person's cognitive state would

be easy and wrong. They are inputs to a threshold, validated only insofar as they let a user drive on purpose.

Environments with obstacles or bystanders. The vehicle has no environmental sensing whatsoever. It cannot see a wall, a stair edge or a foot. Every operating procedure we wrote assumes a clear area and a person holding the kill switch, and that assumption carries real weight.

Chapter 5

Conclusion and future work

5.1 Summary and lessons

NeuroDrive closes a consumer EEG signal onto a moving vehicle in a way that survives the failure of almost every component in the chain. Four safety layers stand in series between a command and a turning wheel, each failing for reasons the others do not share, and the outermost contains no software at all. The protocol, the state machine and the module interfaces are fixed in one contract, and the bridge runs against synthetic and recorded signal sources, so the whole chain can be exercised with no hardware attached.

Some of what we learned generalises past this project.

The interface contract did more for us than anything else we wrote. A few pages beat every tool we tried, because the firmware and the bridge were built by different people in different languages and still integrated without a rewrite.

Simulators pay for themselves in the first week. Writing the state machine a second time in Python felt like duplicated effort, and it made the critical path independent of hardware availability.

Two constants in two repositories will drift. The coupling between the bridge's turn re-send window and the firmware's pulse length was documented, understood, and enforced by nothing, and a change to one of them produced doubled turns until we caught it. A rule that lives only in prose is a rule that will be broken.

Measuring honestly is more useful than measuring favourably. The delay a user feels is dominated by the headset, which reports once per second, and knowing that redirects effort. Optimising our own code path improves nothing a user can perceive.

5.2 Limitations

The vehicle has no environmental sensing and cannot detect an obstacle, a stair edge or a person. The attention metric is a proprietary black box, so we cannot reason about behaviour below the level of the reported number, and the steering gesture is a hand raise detected by a camera, so the vehicle is not steered by thought in any meaningful sense. Latency is dominated by the sensor. Our usability impressions come from the people who built the system, which is the weakest possible sample.

5.3 Future work

Close the remaining cross-repository invariants. This is the immediate item. The startup validator should check the turn repeat window against the firmware's pulse length, and the re-send interval against the watchdog timeout. Better still, the firmware could report its own constants at boot so the bridge verifies real values instead of a copy of them.

Give the vehicle a reflex. A pair of distance sensors and a stop-on-obstacle rule in the firmware would add a fifth safety layer with a genuinely independent trigger, one depending on neither the link, the laptop nor the headset. Of everything on this list, it would most widen the set of environments the vehicle can be operated in.

Steer with a directional signal. A two-class classifier over the raw EEG waveform the headset already streams would make direction a cortical decision, at the cost of a training session for each user. The webcam channel already demonstrates the alternative, an explicitly directional gesture.

Continuous speed, and a user study. Mapping attention above the threshold onto duty cycle rather than a binary go or stop would use information the system currently throws away. A dozen participants on a standard course, measuring time-to-competence and false-start rate, would replace our impressions with evidence, and the session logging needed to support it already exists.

5.4 Concluding remarks

Whether concentration could make a vehicle move was never the interesting part. That took an afternoon. The interesting part was everything that follows from putting a slow, noisy, occasionally absent signal in charge of something with wheels. Stopping has to be more reliable than starting. Components have to be arranged so that no single one has to be trusted to be alive. And the behaviours most worth verifying are exactly the ones a successful demonstration never touches. None of that is specific to EEG. It is what a cyber-physical system demands of the people building it.

Bibliography

- [1] Carlson, T. and Millán, J. d. R. “Brain-controlled wheelchairs: a robotic architecture”. In: *IEEE Robotics & Automation Magazine* 20.1 (2013), pp. 65–73. DOI: 10.1109/MRA.2012.2229936.
- [2] Galán, F., Nuttin, M., Lew, E., Ferrez, P. W., Vanacker, G., Philips, J., and Millán, J. d. R. “A brain-actuated wheelchair: asynchronous and non-invasive brain-computer interfaces for continuous control of robots”. In: *Clinical Neurophysiology* 119.9 (2008), pp. 2159–2169. DOI: 10.1016/j.clinph.2008.06.001.
- [3] Rieiro, H., Díaz-Piedra, C., Morales, J. M., Catena, A., Romero, S., Roca-Gonzalez, J., Fuentes, L. J., and Di Stasi, L. L. “Validation of Electroencephalographic Recordings Obtained with a Consumer-Grade, Single Dry Electrode, Low-Cost Device: A Comparative Study”. In: *Sensors* 19.12 (2019), p. 2808. DOI: 10.3390/s19122808.
- [4] Wolpaw, J. R., Birbaumer, N., McFarland, D. J., Pfurtscheller, G., and Vaughan, T. M. “Brain-computer interfaces for communication and control”. In: *Clinical Neurophysiology* 113.6 (2002), pp. 767–791. DOI: 10.1016/S1388-2457(02)00057-3.
- [5] Zander, T. O. and Kothe, C. “Towards passive brain-computer interfaces: applying brain-computer interface technology to human-machine systems in general”. In: *Journal of Neural Engineering* 8.2 (2011), p. 025005. DOI: 10.1088/1741-2560/8/2/025005.